Guide d'Utilisation - 36TB Intelligence

- 📖 Guide d'Utilisation — 36TB Intelligence
  - Table des matières
  - 1. À qui s'adresse ce guide
  - 2. Ce que fait l'application
  - 3. Prérequis
  - 4. Installation
  - 5. Premiers pas en 10 minutes
  - 6. Où sont vos données
  - 7. Profils de lancement
  - 8. L'interface graphique
  - 9. La CLI `pis`
  - 10. Le scan : politique et sécurité
  - 11. L'extraction de contenu
  - 12. La recherche
  - 13. Couverture sémantique et embeddings de fond
  - 14. Doublons, versions et documents liés
  - 15. Intelligence, timeline, graphe et galaxie
  - 16. Rapports et dossiers
  - 17. Sauvegarde et restauration
  - 18. API locale et MCP
  - 19. Confidentialité et sécurité
  - 20. Dépannage
  - 21. Limites connues
  - 22. Mise à jour, migration, désinstallation
  - 23. Annexes

📖 Guide d'Utilisation — 36TB Intelligence

Version 1.0.0-rc2 (MVP) — Novembre 2026 *Système personnel de connaissances : indexer, rechercher, comprendre et exploiter un disque local — sans envoyer vos documents sur Internet.*

> Ce guide décrit **l'application canonique** : une seule UI Streamlit, une seule CLI (`pis`), une seule base SQLite, un seul store vectoriel local. Évidence réelle : `REAL_CORPUS.md` · Test d'installation neuve : `ACCEPTANCE_NEW_USER.md` · Limites : `RELEASE_NOTES.md` .

Table des matières

1. À qui s'adresse ce guide
2. Ce que fait l'application
3. Prérequis
4. Installation
5. Premiers pas en 10 minutes
6. Où sont vos données
7. Profils de lancement (LITE / SMART / FULL)
8. L'interface graphique
9. La CLI `pis`
10. Le scan : politique et sécurité
11. L'extraction de contenu
12. La recherche
13. Couverture sémantique et embeddings de fond
14. Doublons, versions et documents liés
15. Intelligence, timeline, graphe et galaxie
16. Rapports et dossiers
17. Sauvegarde et restauration
18. API locale et MCP
19. Confidentialité et sécurité
20. Dépannage
21. Limites connues
22. Mise à jour, migration, désinstallation
23. Annexes

---

1. À qui s'adresse ce guide

À toute personne qui possède un gros volume de fichiers personnels (documents, PDF, emails, archives, photos de documents) et veut :

- savoir **ce qu'elle possède** (inventaire, doublons, ancienneté) ;
- **retrouver** un document par son nom, son contenu ou son sens ;
- **comprendre** un corpus (langues, entités, catégories, PII) ;
- **produire** un dossier ou un rapport reproductible.

L'application est **mono-utilisateur**, **locale** et **Linux-first**. Elle ne modifie jamais vos fichiers sources.

---

2. Ce que fait l'application

Domaine	Capacité
Inventaire	Scan rapide et incrémental, cycle de vie (actif/manquant), volumes
Contenu	Extraction PDF, Office/ODF, texte, emails, EPUB, CHM, archives
Recherche	Lexicale (FTS5), sémantique (embeddings locaux), rerank
Compréhension	Langue, entités, catégories, PII (masquée), timeline
Relations	Doublons exacts, quasi-doublons, versions, documents liés, graphe
Exploration	Galaxie sémantique 2D, clusters, topics, contexte documentaire
Décision	Dossiers, 8 types de rapports, export HTML/JSON/PDF avec citations
Exploitation	CLI `pis`, API loopback, doctor, maintenance, sauvegarde/restauration
Confidentialité	100 % local par défaut, politique distante `never`, PII masquée

---

3. Prérequis

- **OS** : Linux (validé). macOS/Windows non validés pour ce MVP.
- **Python** : 3.11 ou 3.12.
- **RAM** : 4 Go minimum ; 16 Go recommandés pour de gros corpus.
- **Disque** : ~1 Go pour l'application + l'espace de la base (≈ 1,4 Ko/fichier indexé) et du store vectoriel.
- **Optionnel** (détecté, jamais requis) : `tesseract` (OCR), `libreoffice` / `soffice` (Office ancien, export PDF), `7z` (CHM), `readpst` (PST/OST), `google-chrome` / `chromium` (PDF alternatif), `ollama` (LLM local).

---

4. Installation

4.1 Depuis les sources (recommandé pour ce dépôt)

```
git clone <votre-dépôt> 36tb-intelligence
cd 36tb-intelligence

python -m venv .venv
source .venv/bin/activate

# Cœur de l'application, versions validées
pip install -c constraints.txt -e .

# Variantes :
# pip install -c constraints.txt -e ".[dev]"        # tests + lint
# pip install -c constraints.txt -e ".[ai,ocr]"     # embeddings locaux, OCR
# pip install -r requirements.txt                   # cœur + extras recommandés
```

4.2 Depuis un wheel

```
python -m venv .venv
source .venv/bin/activate
pip install 36tb_intelligence-1.0.0rc2-py3-none-any.whl
pip check
```

4.3 Vérifier l'installation

```
pis --help
pis doctor
```

`pis doctor` affiche `OK`, `WARN`, `NOT_CONFIGURED` ou `ERROR` pour chaque dépendance et chemin. Un `WARN` n'empêche pas l'utilisation.

> **Important** : le cœur installé sans l'extra `ai` ne contient pas `torch` / `sentence-transformers`. La recherche lexicale fonctionne ; la recherche sémantique et les clusters affichent `NOT_CONFIGURED` avec la commande d'installation à exécuter.

---

5. Premiers pas en 10 minutes

```
# 1. État de départ + base résolue (affichée par toutes les commandes)
pis status

# 2. Scanner un dossier réel (scan partiel sûr par défaut)
pis scan ~/Documents

# 3. Extraire le contenu (par lots bornés)
pis extract --limit 500

# 4. Chercher
pis search facture
pis search "facture impayée client" --mode semantic --limit 10

# 5. Lancer l'interface
scripts/run_full.sh          # ou : pis ui full
# → http://127.0.0.1:8501
```

Parcours conseillé ensuite :

```
pis intel --limit 500          # langues, entités, catégories, PII
pis hashes                     # doublons exacts
pis embed                      # embeddings locaux (couverture sémantique)
pis hashes --near              # quasi-doublons (utilise les vecteurs)
pis clusters --build           # clusters + topics
pis galaxy --limit 1000        # carte sémantique
pis report build --kind SEARCH --query '{"query":"facture"}' --formats HTML,JSON
```

---

6. Où sont vos données

Tous les chemins sont **absolus et indépendants du répertoire courant**.

Donnée	Chemin par défaut	Variable
Base SQLite	`~/.local/share/pis/indexes/files.db`	`PIS_DB_PATH` / `PIS_DATA_DIR`
Store vectoriel + cache modèles	`~/.local/share/pis/cache/embeddings`	`PIS_EMBEDDING_STORE_DIR`
Rapports exportés	`~/.pis-exports`	`PIS_EXPORT_DIR`
Sauvegardes	`~/.pis-backups`	`PIS_BACKUP_DIR`
État des jobs (embedding)	`~/.local/state/pis`	`PIS_STATE_DIR`
Cache général	`~/.cache/pis`	`PIS_CACHE_DIR`
Logs	`~/.local/share/pis/logs`	—

Chaque commande affiche la base utilisée sur `stderr` (et dans `_meta` avec `--json`). Un `--db /chemin/essai.db` permet de travailler sur une base de test sans toucher à la base principale.

Migration d'une ancienne installation

Si vous aviez un dossier `./data` (versions antérieures) :

```
pis migrate-data --dry-run     # plan, sans rien modifier
pis migrate-data               # copie cohérente puis déplacement
```

La migration déplace la base, les rapports et le store vectoriel, et lie la base au store migré. Pour la désactiver : `PIS_NO_AUTO_MIGRATE=1`.

---

7. Profils de lancement

Une seule application, trois profils (mêmes pages cœur, capabilities avancées exposées différemment) :

Profil	Usage	Port par défaut
LITE ⚡	Démarrage minimal, ressources réduites, IA locale	8510
SMART 🎯	Profil adapté au matériel, capacités à la demande	8504
FULL 💪	Toutes les capacités, chargement paresseux	8501

```
scripts/run_lite.sh
scripts/run_smart.sh
scripts/run_full.sh

# équivalent
python -m src.launcher --profile lite
pis ui full --port 8600
```

Le port demandé échoue proprement s'il est occupé ; sinon un port libre proche est choisi. Le launcher affiche le profil, l'URL, la politique distante et la base résolue.

---

8. L'interface graphique

L'interface canonique (`src/ui/app.py`) expose jusqu'à 20 pages selon le profil :

Page	Rôle
🚀 **Scanner**	Lancer un scan, suivre la progression, choisir les exclusions
🔍 **Search**	Recherche lexicale, filtres, ouverture de fichiers
🎯 **Advanced Search**	Filtres avancés, tri, suggestions
🧬 **Duplicates & Versions**	Hachage, doublons exacts/quasi, versions, liés, rerank
🧠 **Document Intelligence**	Langue, entités, catégories, PII (masquée)
🧭 **Timeline & Graph**	Chronologie, voisinage, priorité
🌌 **Galaxy & Topics**	Carte sémantique, clusters, topics, contexte
🛠️ **Ingestion & Coverage**	File d'extraction, échecs, couverture
📁 **Dossiers & Reports**	Créer, prévisualiser, exporter, historique
🧰 **Operations & System**	Doctor, maintenance, sauvegarde/restauration
🧠 **AI Search** / 💬 **AI Chat**	Recherche et conversation locales (SMART/FULL)
🏷️ **Tags & Favorites**	Organisation manuelle
📊 **Dashboard** / 📈 **Statistics**	Vue d'ensemble et métriques
🌌 **Visualizations**	Graphiques Plotly/NetworkX (SMART/FULL)
👁️ **File Viewer**	Aperçu, archives, actions
🔄 **Auto-Extract**	Extraction automatique bornée (SMART/FULL)
🤖 **Advanced AI**	Résumés, Q&A, recommandations (SMART/FULL)
☁️ **Cloud Sync**	Synchronisation manuelle (SMART/FULL)

La barre latérale affiche aussi :

- **Couverture sémantique X %** (`embedded / eligible`) et la couverture d'extraction ;
- l'état du job d'embedding avec ▶️ **Embeddings** et ⏹️ **Stop** ;
- le statut du backend IA, le profil actif et la politique distante.

Scanner depuis l'interface

1. Sélectionnez un ou plusieurs chemins.
2. Laissez cochée « **Continuer malgré les dossiers illisibles** » (mode sûr).
3. Ajoutez éventuellement des exclusions (`node_modules` , `*.tmp` , `/mnt/backup`).
4. Lancez. Un scan partiel se termine en **COMPLETED_WITH_WARNINGS** et affiche les périmètres non parcourus ; la réconciliation n'y touche pas.

---

9. La CLI `pis`

9.1 Options globales

```
pis [--db CHEMIN] [--json] [-v|--verbose] [-q|--quiet] <commande> [options]
```

Ces options sont aussi acceptées **après** la sous-commande (`pis scan ~/Documents --json`).

Option	Effet
`--db CHEMIN`	Base explicite (absolue de préférence)
`--json`	Sortie machine, avec `_meta` (base, data dir, commande)
`-v`	Logs `DEBUG` sur `stderr`
`-q`	Logs `ERROR` seulement

9.2 Codes de sortie

Code	Signification
0	Succès complet
3	Succès partiel : scan avec périmètres non parcourus, opération annulée, backend indisponible
2	Échec, argument invalide, erreur d'exécution

9.3 Commandes

Commande	Rôle	Exemple
`status`	Santé agrégée, couverture, politique	`pis status --json`
`doctor`	Diagnostic détaillé	`pis doctor --deep`
`scan`	Scanner une racine	`pis scan ~/Documents`
`extract`	Extraire le contenu	`pis extract --limit 500`
`retry`	Rejouer les échecs éligibles	`pis retry --limit 100`
`ingest-issues`	Lister les problèmes d'extraction	`pis ingest-issues`
`search`	Rechercher	`pis search "contrat" --mode rerank`
`semantic-refresh`	Rafraîchissement borné au premier plan	`pis semantic-refresh --limit 256`
`embed`	Job d'embedding reprenable	`pis embed --background`
`hashes`	Hachage exact + doublons	`pis hashes --near`
`derived`	Couches dérivées en une passe	`pis derived --only hashes,intel`
`duplicates`	Statistiques doublons/versions	`pis duplicates`
`intel`	Langue, entités, catégories, PII	`pis intel --limit 500`
`graph`	Voisinage ou graphe d'entités	`pis graph --file-id 12 --depth 2`
`galaxy`	Projection sémantique 2D	`pis galaxy --limit 1000 --persist`
`clusters`	Construire/lister clusters et topics	`pis clusters --build --k 12`
`topics`	Lister les topics	`pis topics`
`context`	Contexte d'un document	`pis context 12`
`dossier`	Gérer les dossiers	`pis dossier create --name "Client X"`
`report`	Construire/lister les rapports	`pis report build --kind SEARCH ...`
`maintenance`	Opérations bornées	`pis maintenance semantic-consistency`
`backup`	Sauvegarde cohérente	`pis backup`
`restore`	Restauration vérifiée	`pis restore <archive> --confirm`
`api`	API loopback	`pis api --port 8600`
`migrate-data`	Migrer `./data` vers XDG	`pis migrate-data`
`ui`	Lancer l'application	`pis ui full`

Chaque commande accepte `-h` pour le détail de ses options.

---

10. Le scan : politique et sécurité

10.1 Mode sûr par défaut (recommandé)

Un dossier illisible **n'annule plus** le scan. L'application :

1. enregistre chaque erreur par chemin (`scan_errors`) ;
2. termine en `COMPLETED_WITH_WARNINGS` (code 3) ;
3. réconcilie uniquement les sous-arbres **intégralement parcourus** ;
4. laisse les fichiers situés sous un dossier illisible ou exclu **inchangés** ;
5. si le nombre de périmètres non parcourus dépasse la limite de sécurité (500), termine quand même en `COMPLETED_WITH_WARNINGS` mais **annule entièrement la réconciliation** : aucun fichier n'est marqué manquant. Un scan de `/` tombe typiquement dans ce cas (zones système protégées) ; pour bénéficier de la réconciliation, scannez un sous-arbre (`/home`) ou ajoutez des exclusions.

```
pis scan ~/Documents
# scan COMPLETED_WITH_WARNINGS: seen=... errors=3 unscanned_scopes=3
```

10.2 Mode strict (fail-closed)

```
pis scan ~/Documents --strict
```

Au premier accès refusé : statut `FAILED` (code 2), **aucune réconciliation**. Adapté aux scripts qui exigent une exhaustivité prouvée.

10.3 Exclusions

```
pis scan /media/disque \
  --exclude node_modules \
  --exclude '*.tmp' \
  --exclude-path /media/disque/sauvegardes

# ou via l'environnement
export PIS_SCAN_EXCLUDE_DIRS="node_modules,*.tmp,/mnt/backup"
pis scan /media/disque
```

Les sous-arbres exclus sont enregistrés et **protégés de la réconciliation**.

10.4 Scan borné (diagnostic)

```
pis scan ~/Documents --limit 1000 --dry-run   # plan seulement
pis scan ~/Documents --limit 1000             # indexe 1000 fichiers, sans réconciliation
```

Un scan borné est toujours `CANCELLED` (code 3) : l'absence des autres fichiers n'est pas interprétée comme une suppression.

10.5 Bonnes pratiques

- Commencez par un dossier, puis élargissez.
- Gardez le mode sûr ; utilisez `--strict` pour des contrôles d'intégrité.
- Sur un disque externe, vérifiez que le point de montage est stable avant un scan complet : la réconciliation dépend de l'identité du volume.
- Un scan sans changement est bon marché (incrémental).

---

11. L'extraction de contenu

```
pis extract --limit 500        # lot borné (max 500 par appel)
pis extract --dry-run          # combien de documents éligibles
pis extract --scope /home/...  # restreindre à un préfixe
pis extract --ocr              # activer l'OCR pour ce lot
pis retry --limit 100          # rejouer les échecs temporaires
pis ingest-issues              # voir la file et les résultats
```

- **Formats** : PDF, DOC/DOCX, XLS/XLSX, PPT/PPTX, ODT/ODS, RTF, TXT/MD/CSV, HTML/XML/JSON/YAML, emails (`.eml` , `.msg`), EPUB, CHM, archives (ZIP, TAR, 7z, RAR selon outils).
- **Limites** : extraction et hachage plafonnés à 50 Mio par fichier ; les fichiers plus gros restent indexés en métadonnées.
- **OCR** : désactivé par défaut (`PIS_OCR_ENABLED=1` pour l'activer ; langues via `PIS_OCR_LANGS` , défaut `fra+deu+eng`). Nécessite `tesseract` .
- **File d'échecs** : chaque résultat est classé (`EXTRACTED` , `NO_TEXT` , `UNSUPPORTED_FORMAT` , `ENCRYPTED` , `PERMANENT_ERROR` , `TIMEOUT` , ...). Les échecs temporaires sont rejouables ; rien n'est supprimé.
- **PST/OST** : nécessite `readpst` / `pypff` ; sinon `NOT_CONFIGURED` , sans échec du pipeline.

---

12. La recherche

```
pis search "facture"                        # lexicale (FTS5)
pis search "facture impayée client" --mode semantic
pis search "contrat" --mode rerank          # hybride + fusion
pis search facture --extension .pdf --limit 20
pis search "" --category finance --language fr
```

Mode	Ce qu'il fait	Prérequis
`lexical`	Nom, chemin, contenu indexé FTS5 ; rapide et exact	aucun
`semantic`	Similarité de sens via embeddings locaux	couverture > 0
`rerank`	Fusion lexicale + sémantique + signaux doublons/versions	couverture > 0

Filtres disponibles : `--extension` , `--document-kind` , `--language` , `--category` , `--limit` .

Dans l'interface, la page **Search** ouvre le fichier ou son dossier, et la page **Advanced Search** ajoute tri, suggestions et filtres.

Pagination des résultats

Les résultats ne sont pas des pages web séparées : la même table affiche la page sélectionnée.

- **Résultats par page** : 20 / 50 / 100 / 200 (choisir 100 ou 200 pour voir tous les résultats d'un coup).
- **← Précédent / Suivant →** ou le curseur **Page** pour changer de page ; les commandes sont disponibles en haut **et** en bas de la liste.
- **Max Results** (au-dessus des résultats) contrôle le nombre total de résultats récupérés par la recherche (10 à 1000). Pour aller au-delà de 100, augmentez cette valeur avant de lancer la recherche.
- Le panneau 👁️ **Détails** s'ouvre sous la ligne sélectionnée, puis se ferme avec ❌ **Fermer**.

---

13. Couverture sémantique et embeddings de fond

La **couverture sémantique** est le ratio :

```
documents embarqués / documents éligibles
(= ACTIVE, contenu extrait > 50 caractères)
```

13.1 Vérifier

```
pis status                # semantic=100.0% (614/614)
pis embed --status        # état du job + couverture
```

13.2 Embarquer

```
# Au premier plan (bloquant, affiche le résultat)
pis embed

# En tâche de fond (reprend après un arrêt ou un redémarrage)
pis embed --background
pis embed --status
pis embed --stop

# Réparer un store perdu/incohérent ou une base migrée
pis embed --rebuild
```

Le job :

- traite uniquement les documents « dirty » (nouveaux ou modifiés) ;
- est **idempotent et reprenable** : un arrêt laisse les lignes à refaire ;

- écrit son état dans `~/.local/state/pis/embedding_job.json` et son journal dans `~/.local/share/pis/logs/embedding_job.log` ;
- n'embarque que les documents dont le contenu est suffisant.

> **Sans l'extra `ai`**, `pis embed` s'arrête proprement en `NOT_CONFIGURED` et indique `pip install -e ".[ai]"`. La recherche lexicale reste disponible.

13.3 Seuils et bon sens

- 614 documents → ~8 s ; 8 384 documents → ~37 s (GPU RTX 3090, all-MiniLM-L6-v2).
- Sur de très gros corpus, lancez `--background` et priorisez par périmètre (`pis scan` / `pis embed --limit`).
- `pis maintenance semantic-consistency` doit afficher `store_vectors == embedded_rows`.

---

14. Doublons, versions et documents liés

```
pis hashes                      # doublons exacts (taille d'abord, SHA-256)
pis hashes --near               # quasi-doublons (vecteurs si disponibles)
pis duplicates                  # statistiques
```

- **Exacts** : même contenu binaire, quel que soit le nom. Aucun fichier n'est déplacé ou supprimé.
- **Quasi-doublons** : LSH + confirmation sémantique, borné (pas de $O(N^2)$).
- **Versions** : familles déduites de marqueurs explicites uniquement ; aucune chronologie inventée.
- **Liés** : suggestions pour la recherche et les rapports.

Dans l'UI : page **Duplicates & Versions** (onglets Exact, Near, Versions, Related, Reranked search). Le bouton de hachage y est disponible ; en CLI, utilisez `pis hashes`.

---

15. Intelligence, timeline, graphe et galaxie

```
pis intel --limit 500           # langue, entités, catégories, PII
pis graph --file-id 12 --depth 2 # voisinage documentaire
pis graph --min-docs 3          # graphe d'entités
pis galaxy --limit 1000 --persist
pis clusters --build --k 12
pis topics
pis context 12
```

- **Langues** : FR/DE/EN prioritaire, détection locale sans dépendance externe ; les fichiers de code sont classés `code`, les cas incertains `und`.

- **Entités** : dates, URL, téléphones, organisations, montants, lieux, emails, personnes (heuristiques explicitement marquées).
- **Catégories** : taxonomie configurable, multi-label, override manuel.
- **PII** : détection validée (email, téléphone, adresse, IBAN, carte, etc.), **stockée masquée + empreinte** uniquement. Aucune valeur brute n'est exportée.
- **Timeline** : sources de dates explicites (`KNOWN` / `DERIVED`), jamais inventées.
- **Galaxie** : projection PCA/SVD bornée ; agrégation pour les gros volumes. C'est une carte exploratoire, pas une distance littérale.
- **Clusters/topics** : MiniBatchKMeans/KMeans + c-TF-IDF déterministe ; libellés appuyés sur des termes du corpus.

---

16. Rapports et dossiers

16.1 Dossiers

```
pis dossier create --name "Client X" --mode STATIC --doc-ids 12 18 44
pis dossier create --name "Veille" --mode DYNAMIC \
  --query '{"kind":"search","query":"réglementation"}'
pis dossier list
pis dossier add --dossier-id dos_... --doc-ids 51
pis dossier freeze --dossier-id dos_...
pis dossier compare --dossier-id dos_...
```

- `STATIC` : liste figée de documents.
- `DYNAMIC` : requête rejouée à chaque construction.
- `freeze` : snapshot comparable dans le temps.

16.2 Rapports

```
pis report build --kind SEARCH \
  --title "Factures 2026" \
  --query '{"query":"facture"}' \
  --formats html,json,pdf \
  --privacy MASK_PII

pis report list
```

Paramètre	Valeurs
`--kind`	`SEARCH`, `DOSSIER`, `TIMELINE`, `DUPLICATES`, `ENTITY_CATEGORY`, `PII_SUMMARY`, `INGESTION`, `PROJECT`
`--formats`	`HTML`, `JSON`, `PDF` (insensible à la casse)
`--privacy`	`FULL_LOCAL`, `MASK_PII`, `OMIT_HIGH_SENSITIVITY`, `PATH_REDACTED`, `METADATA_ONLY`
`--doc-ids`	Identifiants explicites
`--query`	JSON de recherche
`--dry-run`	Construit l'IR sans écrire d'artefact

Chaque rapport produit :

- un **HTML autonome** (CSS inline, aucune ressource distante) ;
- un **JSON** structuré ;
- un **PDF** si un fournisseur local est disponible (LibreOffice/Chrome) ;
- un **manifeste** (empreinte logique, sources, mode de confidentialité, avertissements).

Les citations utilisent `[D12]` / `[D12:p3]` avec provenance `KNOWN` / `DERIVED` / `INFERRED` / `UNAVAILABLE`. Aucune page n'est inventée.

> Les artefacts sont écrits dans `~/.pis-exports` (ou `PIS_EXPORT_DIR`), jamais dans le corpus scanné.

---

17. Sauvegarde et restauration

```
# Sauvegarde applicative cohérente (SQLite + store vectoriel + config)
pis backup
pis backup --out /media/backup --no-semantic

# Vérifier sans restaurer
pis restore /media/backup/pis-backup-....tar.gz --verify-only

# Restaurer (staging + intégrité + copie de rollback)
pis restore /media/backup/pis-backup-....tar.gz --confirm

# Cibler une autre base
pis restore <archive> --target-db /tmp/restore.db --confirm
```

La sauvegarde utilise l'API de backup SQLite (cohérente même en WAL), vérifie les empreintes, et ne copie **aucun secret**. La restauration refuse d'écraser silencieusement et conserve une copie `.pre-restore`.

---

18. API locale et MCP

18.1 API REST (loopback uniquement)

```
pis api                    # http://127.0.0.1:8600/api/v1
pis api --port 8610
```

Principaux points d'entrée (lecture, PII masquée, chemins expurgeables) :

```
GET  /api/v1/health
GET  /api/v1/doctor
GET  /api/v1/files?limit=25
GET  /api/v1/files/{id}
GET  /api/v1/search?q=facture&mode=lexical
GET  /api/v1/duplicates
GET  /api/v1/entities
GET  /api/v1/categories
GET  /api/v1/timeline
GET  /api/v1/graph/neighborhood/{id}
GET  /api/v1/ingestion/issues
GET  /api/v1/dossiers
GET  /api/v1/reports
GET  /api/v1/galaxy
GET  /api/v1/clusters
GET  /api/v1/topics
GET  /api/v1/context/{id}
```

L'API est **mono-utilisateur, sans authentification ni TLS**, et refuse par défaut une écoute non-loopback (`--allow-remote` explicite, déconseillé).

18.2 MCP (lecture seule, stdio)

```
python -m src.mcp_server
```

Trois outils : `search_files`, `get_document`, `semantic_search` (repli lexical si les embeddings sont absents). Aucune écriture, aucun scan.

---

19. Confidentialité et sécurité

- **Aucune télémétrie**.

- **Politique de contenu distant** : `PIS_REMOTE_CONTENT_POLICY=never` par défaut dans tous les profils. Un appel distant exige une configuration explicite (mode `hybrid` / `api`, politique `extracted_text`, clé API).
- **PII** : détection locale, stockage masqué + empreinte salée ; les API et exports masquent par défaut (`MASK_PII` recommandé).
- **Aucune modification des source** : le scan est en lecture seule ; l'extraction n'écrit que dans la base.
- **Sorties hors corpus** : exports et sauvegardes vont dans des répertoires dédiés.
- **API** : loopback uniquement, pas d'auth/TLS (mono-utilisateur).
- **Chiffrement au repos** : non fourni au niveau applicatif ; utilisez le chiffrement disque de l'OS.

---

20. Dépannage

Symptôme	Cause probable	Solution
`Permission denied` pendant le scan	Sous-arbre illisible	Mode sûr par défaut ; sinon `--ignore-errors`, `--exclude`, ou lancer avec des droits adaptés
`scan FAILED` (code 2)	`--strict` + accès refusé	Relancer sans `--strict` ou exclure le sous-arbre
`scan COMPLETED_WITH_WARNINGS` (code 3)	Périmètres non parcourus	Normal en mode sûr ; vérifier `unscanned_scopes`
« réconciliation entièrement annulée »	Plus de 500 périmètres illisibles/exclus (typique d'un scan de `/`)	Comportement sûr : les fichiers lisibles sont indexés, aucun `MISSING`. Scanner un sous-arbre (`/home`) ou exclure les zones système pour réconcilier
`semantic=0.0%`	Aucun document extrait ou extrait trop court	`pis extract`, puis `pis embed`
`NOT_CONFIGURED` pour l'embedding	Extra `ai` absent	`pip install -c constraints.txt -e ".[ai]"`
Recherche sémantique sans résultat	Couverture insuffisante	`pis embed`, vérifier `pis embed --status`
`store_vectors != embedded_rows`	Store incohérent / base migrée	`pis embed --rebuild` puis `pis maintenance semantic-consistency`
PDF non généré	Aucun fournisseur local	Installer LibreOffice ou Chrome, ou exporter en HTML/JSON
`pis report` → 0 artefact	Format invalide	Utiliser `HTML`, `JSON`, `PDF` (la casse est tolérée)
`port déjà utilisé`	Autre instance	`--port` libre ou fermer l'instance
`base introuvable` / résultats vides	Base différente	`pis status` affiche la base ; utiliser `--db` ou `PIS_DATA_DIR`
Anciennes données absentes	Non migrées	`pis migrate-data` (ou `PIS_DATA_DIR=./data` temporairement)
Ollama indisponible	Daemon arrêté / modèle absent	`ollama serve`, `ollama pull <modèle>` ; sinon mode local lexical
OCR sans effet	OCR désactivé	`PIS_OCR_ENABLED=1` + `tesseract` installé, puis `pis extract --ocr`
Fichier ignoré	Extension/exclusion	Vérifier la politique du scanner et `--exclude`
Embedding très long	Gros corpus / modèle lourd	`pis embed --background`, `--limit`, ou périmètre réduit

Logs utiles :

```
pis -v status                # DEBUG sur stderr
pis -q scan ~/Documents      # erreurs seulement
tail -f ~/.local/share/pis/logs/embedding_job.log
```

---

21. Limites connues

- **Échelle 36 To** : conçue, pas encore mesurée de bout en bout. Validé sur 15 192 fichiers / 614 documents embarqués, et 8 384 documents sur une base réelle de 1,76 M fichiers (voir `REAL_CORPUS.md`).
- **OS** : Linux validé ; macOS/Windows non validés pour ce MVP.
- **Dette statique** : ruff/mypy sont sous budget décroissant, pas à zéro.
- **PostgreSQL/pgvector** : optionnel, non exercé ; le backend canonique est le store NumPy local.
- **PST/OST** : outils externes requis.
- **API** : pas d'authentification/TLS (mono-utilisateur, loopback).
- **Versions** : familles déduites uniquement de marqueurs explicites.

---

22. Mise à jour, migration, désinstallation

Mise à jour

```
source .venv/bin/activate
pip install -c constraints.txt -e . --upgrade
pis doctor
```

La base est migrée automatiquement (colonnes additives). Faites une sauvegarde avant une mise à jour majeure :

```
pis backup
```

Réinitialiser l'index (sans toucher aux sources)

```
pis --db ~/.local/share/pis/indexes/files.db maintenance integrity
# ou repartir d'une base neuve :
pis --db /tmp/nouvelle.db scan ~/Documents
```

Ne supprimez jamais `~/.local/share/pis` sans sauvegarde : la base contient l'index, les rapports et l'historique.

Désinstallation

```
pip uninstall 36tb-intelligence
# optionnel : supprimer les données (IRRÉVERSIBLE)
rm -rf ~/.local/share/pis ~/.pis-exports ~/.pis-backups ~/.local/state/pis
```

---

23. Annexes

23.1 Variables d'environnement

Variable	Défaut	Rôle
`PIS_DATA_DIR`	`~/.local/share/pis`	Racine des données
`PIS_DB_PATH`	`<data>/indexes/files.db`	Base SQLite
`PIS_EMBEDDING_STORE_DIR`	`<data>/cache/embeddings`	Store vectoriel
`PIS_EXPORT_DIR`	`~/.pis-exports`	Rapports
`PIS_BACKUP_DIR`	`~/.pis-backups`	Sauvegardes
`PIS_STATE_DIR`	`~/.local/state/pis`	État des jobs
`PIS_CACHE_DIR`	`~/.cache/pis`	Cache général
`PIS_NO_AUTO_MIGRATE`	non	Désactive la migration automatique
`PIS_SCAN_IGNORE_ERRORS`	`1`	Scan partiel sûr
`PIS_SCAN_EXCLUDE_DIRS`	—	Exclusions (noms, globs, chemins)
`PIS_REMOTE_CONTENT_POLICY`	`never`	Contenu distant
`PIS_AI_MODE`	`auto`	`local` , `hybrid` , `api` , `auto`
`PIS_OCR_ENABLED`	`0`	Active l'OCR
`PIS_OCR_LANGS`	`fra+deu+eng`	Langues OCR
`PIS_LAUNCH_PROFILE`	`smart`	Profil par défaut
`PIS_API_HOST` / `PIS_API_PORT`	`127.0.0.1` / `8600`	API
`PIS_LOG_LEVEL`	`WARNING`	Niveau de log
`PIS_MCP_TRANSPORT`	`stdio`	Transport MCP

23.2 Codes de sortie

`0` succès · `3` partiel/annulé · `2` échec.

23.3 Extensions traitées (extraction)

`.pdf .doc .docx .xls .xlsx .ppt .pptx .odt .ods .rtf .txt .md .csv .html .htm .xml .json .yaml .yml .py .js .css .sql .sh .ps1 .bat .cfg .ini .log .eml .msg .epub .chm .wpd .wps` — plus les archives (ZIP/TAR/7z/RAR) et les images/PDF via OCR lorsqu'il est activé.

23.4 Glossaire

- **ACTIVE / MISSING** : état de cycle de vie d'un fichier indexé.
- **Réconciliation** : passage d'un scan complet qui associe les renommages et marque les disparitions ; jamais exécutée sur un scan partiel/échoué.
- **Scope non parcouru** : sous-arbre illisible ou exclu, protégé de la réconciliation.
- **Couverture sémantique** : `documents embarqués / documents éligibles`.
- **Embedding** : vecteur numérique représentant le sens d'un document.
- **Rerank** : fusion lexicale + sémantique + signaux de doublons/versions.
- **Dossier** : sélection statique ou dynamique de documents.
- **IR** : représentation intermédiaire d'un rapport (sections, sources, provenance) avant export.

23.5 Où aller plus loin

- Preuves réelles : `REAL_CORPUS.md`
- Test d'installation neuve : `ACCEPTANCE_NEW_USER.md`
- Capacités et limites : `RELEASE_NOTES.md`
- Installer/exploiter : `RELEASE_CANDIDATE.md`
- Protocole qualité : `VALIDATION_PROTOCOL.md`

Guide généré pour 36TB Intelligence v1.0.0-rc2. Les commandes et chemins correspondent à l'implémentation validée du dépôt.